CS 473 (Spring 2025) Yiming Qin (yimingq3@illinois.edu)

Problem 1.1

Solution:

(a)

Algorithm 1

1: Define a list of all rectangles R, rectangles are represented as (y, x_1, x_2).
2: Define a list of all points P, points are represented as (y, x).
3: Merge P and R into I, sort according to y-coord. ▷ $O(n \log n)$
4: Maintain one point on each side that is the closest to the vertical line, p_l and p_r.
5: **for** i in I **do** ▷ $O(n)$
6: **if** i is point **then**
7: check if the point is closer to ℓ, if so, update p_l or p_r.
8: **else**
9: check if p_l or p_r is inside the rectangle.
10: **end if**
11: **end for**

Total time complexity is $O(n \log n)$.

(b)

Algorithm 2

1: Define a list of all rectangles R, rectangles are represented as (y, x_1, x_2).
2: Define a list of all points P, points are represented as (y, x).
3: Merge P and R into I, pre-sort I according to y-coord. ▷ $O(n \log n)$
4: Sort I according to x_1-coord of rectangles and x-coord of points. ▷ $O(n \log n)$
5: Compute vertical line ℓ, x_ℓ = ($I[0]$ + $I[I.\text{length - } 1]$) / 2
6: Maintain a boolean list b to keep track of rectangles that are crossed by the vertical line.
7: Get the pre-sorted sublist of all rectangles that are crossed with the line and all points in the subarea, call solution in part(a). ▷ $O(n \log n)$ in total
8: If there are no empty rectangles, split the subarea into 2 parts by the vertical line, each containing only uncrossed rectangles and points.
9: Recursively process each subarea from step 5.

Total time complexity is $O(n \log n)$.